

M,kModule- 4 (Recurrent Neural Network)

Recurrent neural networks – Computational graphs, RNN design, encoder – decoder sequence to sequence architectures, deep recurrent networks, recursive neural networks, modern RNNs LSTM and GRU.

The few issues in the feed-forward neural network are

- Cannot handle sequential data
- Considers only the current input
- Cannot memorize previous inputs

The solution to these issues is the recurrent neural networks (RNN)

A Deep Learning approach for modelling sequential data is **Recurrent Neural Networks (RNN)**. Recurrent Neural Networks use the same weights for each element of the sequence, decreasing the number of parameters. Recurrent neural networks were first developed in the 1980s. The advent of long short-term memory (LSTM) in the 1990s, combined with an increase in computational power and the vast amounts of data to deal with, has pushed RNNs to the forefront.

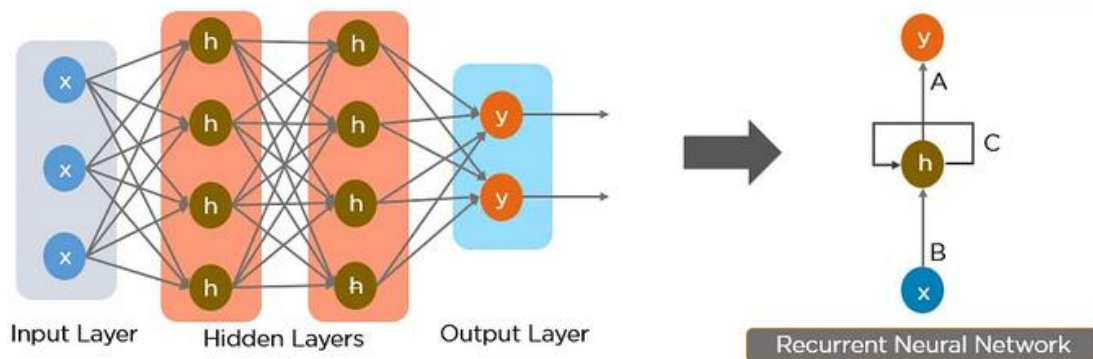
Sequence data are the data points which are ordered in the meaningful manner such that earlier data points or observations provide the information about later data points or observations. Time-series data is a sequence of data points collected over time intervals, allowing us to *track changes over time*. Time-series data can track changes over milliseconds, days, or even years. Eg: daily highs and lows in temperature, opening and closing value of the stock market, daily hospitalizations due to COVID-19.

RNN DESIGN

All of the inputs and outputs in standard neural networks are independent of one another, however in some circumstances, such as when predicting the next word of a phrase, the prior words are necessary, and so the previous words must be remembered. As a result, RNN was created, which used a hidden Layer to overcome the problem. The most important component of

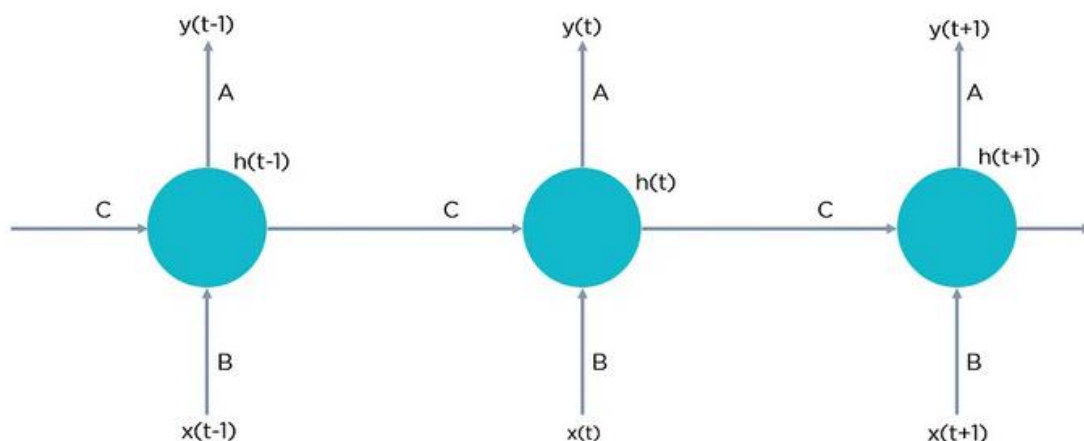
RNN is the Hidden state, which remembers specific information about a sequence. RNNs have a memory that stores all information about the calculations.

Apple's Siri and Google's voice search both use Recurrent Neural Networks.



Here, “x” is the input layer, “h” is the hidden layer, and “y” is the output layer. A, B, and C are the parameters of the network to improve the output of the model.

At any given time t , the current input is a combination of input at $x(t)$ and $x(t-1)$. The output at any given time is fetched back to the network to improve on the output.



$$h(t) = f_c(h(t-1), x(t))$$

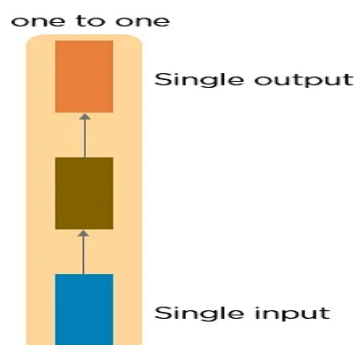
$h(t)$ = new state
 f_c = function with parameter c
 $h(t-1)$ = old state
 $x(t)$ = input vector at time step t

Types of Recurrent Neural Networks

There are four types of Recurrent Neural Networks:

1. One to One
2. One to Many
3. Many to One
4. Many to Many

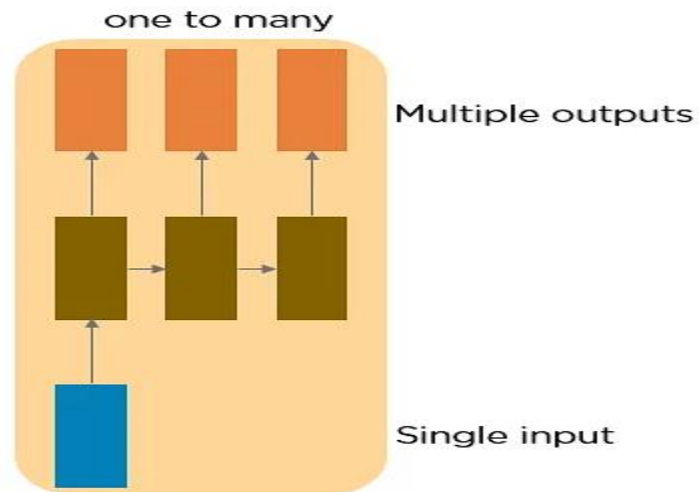
One to One RNN-This type of neural network is known as the Vanilla Neural Network. It's used for general machine learning problems, which has a single input and a single output. A one-to-one architecture is used in traditional neural networks.



One to Many RNN-This type of neural network has a single input and multiple outputs.

An example is image captioning **where the description of an image is generated**. The image captioning model consists of an encoder and a decoder. The encoder extracts out important features from the image. The decoder takes those features as inputs and uses them to generate the caption.

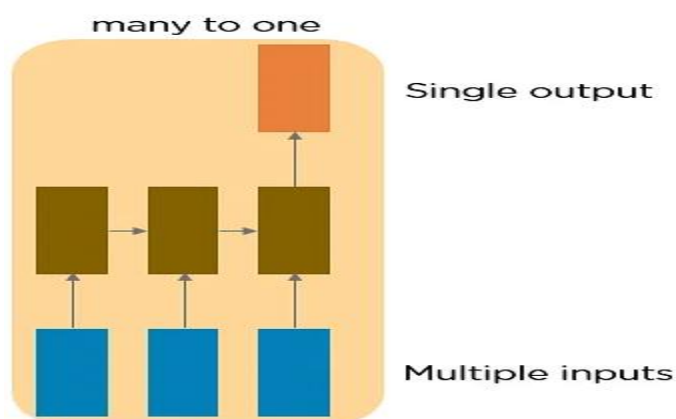
Another example is the production of music.



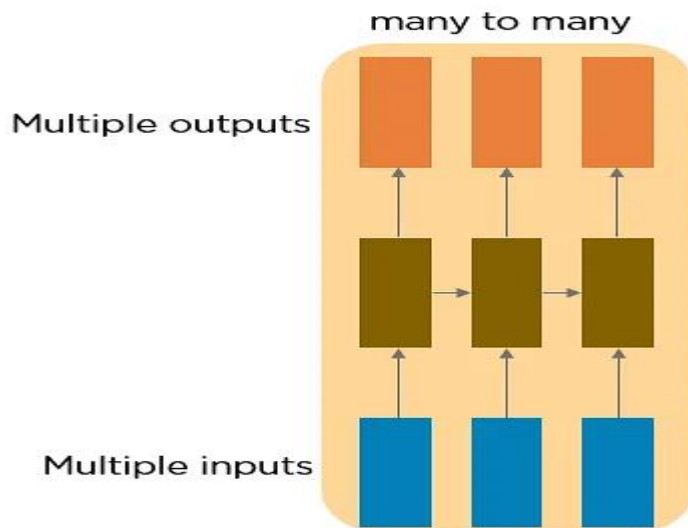
Many to One RNN--This RNN takes a sequence of inputs and generates a single output. Sentiment analysis is a good example of this kind of network where a given sentence can be classified as expressing positive, negative or neutral sentiments.

For example: "I really like the new design of your website!" → **Positive**.

Sentiment analysis studies the subjective information in an expression, that is, the opinions, appraisals, emotions, or attitudes towards a topic, person or entity.



- **Many to Many RNN**--This RNN takes a sequence of inputs and generates a sequence of outputs. Machine translation systems use many to many networks.



Computational graphs

Computational graph is a directed graph that is used for expressing and evaluating mathematical expressions. These can be used for two different types of calculations:

- Forward computation
- Backward computation

The key terminologies in computational graphs are

- A variable is represented by a **node** in a graph. It could be a scalar, vector, matrix, tensor etc.
- Function argument and data dependencies are both represented by an **edge**. These are similar to node pointers.
- A simple function of one or more variables is called an **operation**. Complex functions can be represented by combining multiple operations.

For example, consider the following example:

$$Y = (a + b) * (b - c)$$

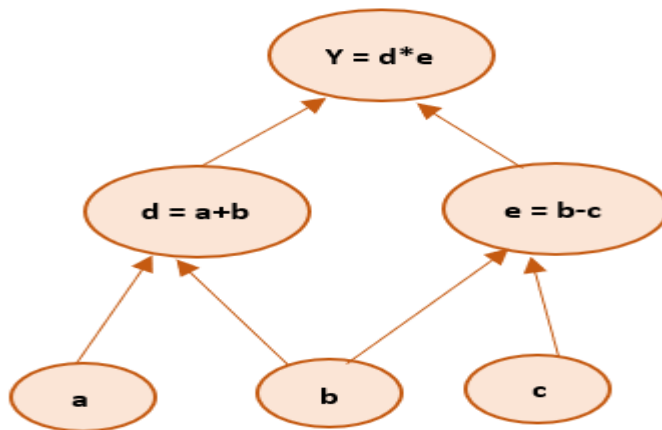
For better understanding, introduce two variables d and e such that every operation has an output variable. We now have:

$$d = a + b$$

$$e = b - c$$

$$Y = d * e$$

Here three operations-addition, subtraction, and multiplication are performed. To create a computational graph, create nodes, each of them has different operations along with input variables.



The final output value is found by initializing input variables and computing nodes of the graph.

Computational Graphs in Deep Learning

Computations of the neural network are organized in terms of a forward pass or forward propagation which computes the output of the neural network, followed by a backward pass or backward propagation step to compute gradients/derivatives.

The back-propagation algorithm is implemented mostly using the idea of a computational graph, where each neuron is expanded to many nodes in the computational graph and performs a simple mathematical operation like addition, multiplication. The computational graph does not have any weights on the edges; all weights are assigned to the nodes. The backward propagation algorithm is then run on the computational graph. Once the calculation is complete, only the gradients of the weight nodes are required for update.

One commonly used optimization function that adjusts weights according to the error they caused is called the “gradient descent.” Gradient is another name for slope, and slope, on an x-y graph, represents how two variables are

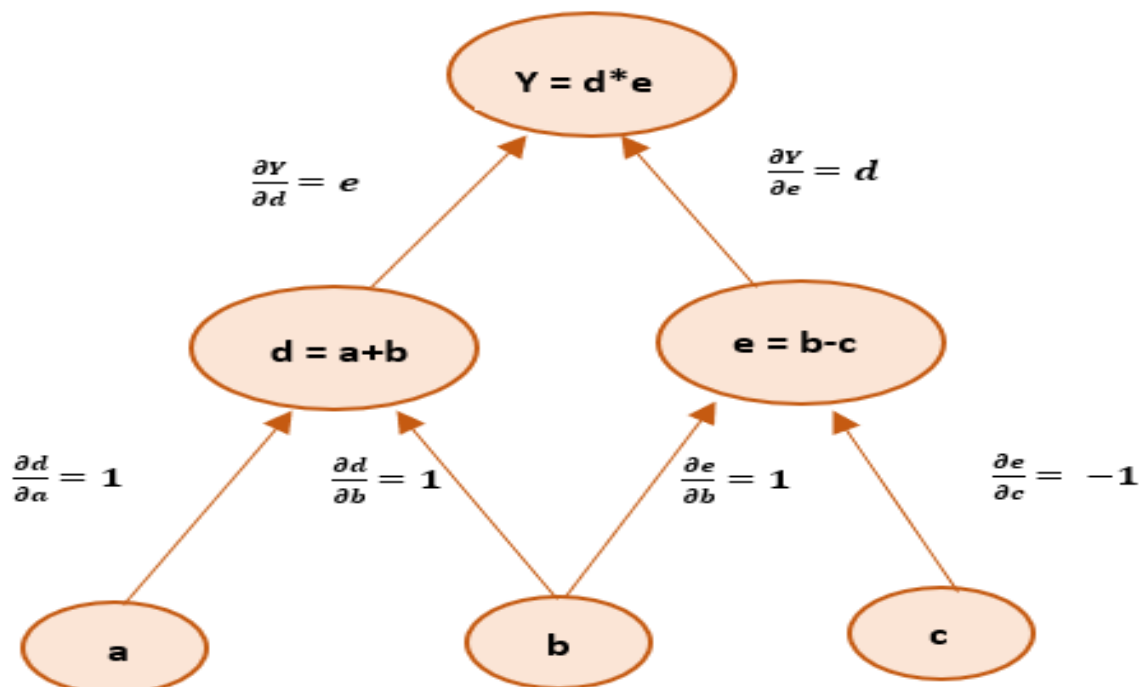
related to each other. In this case, the slope is the ratio between the network's error and weight; i.e., how does the error change as the weight is varied or finds which weight produce the least error.

Each weight is just one factor in a deep network that involves many transforms; the signal of the weight passes through activations and sums over several layers, so use the chain rule of calculus to work back through the network activations and outputs.

Given two variables, error and weight, mediated by a third variable, **activation**, through which the weight is passed. To calculate how a change in weight affects a change in error, firstly calculate how a change in activation affects a change in Error, and how a change in weight affects a change in activation.

To understand derivatives in a computational graph, understand how a change in one variable brings change on the variable that depends on it. If **a** directly affects **c**, then find out how it affects c. If a slight change in the value of **a** occurs, how does **c** change? This is termed as the partial derivative of c with respect to a.

Graph for backpropagation to get derivatives will look like this:



We have to follow chain rule to evaluate partial derivatives of final output variable with respect to input variables: a, b, and c. Therefore the derivatives can be given as :

$$\frac{\partial Y}{\partial a} = \frac{\partial Y}{\partial d} \times \frac{\partial d}{\partial a} = e \times 1 = e$$

$$\frac{\partial Y}{\partial b} = \frac{\partial Y}{\partial d} \times \frac{\partial d}{\partial b} = e \times 1 = e$$

$$\frac{\partial Y}{\partial c} = \frac{\partial Y}{\partial e} \times \frac{\partial e}{\partial c} = d \times -1 = -d$$

This gives us an idea of how computational graphs make it easier to get the derivatives using back propagation.

Types of computational graphs:

- Static Computational Graphs
- Dynamic Computational Graphs

Static Computational Graphs- To train the model and generate predictions, a lot of data is required. The benefits are powerful graph optimization and scheduling.

Dynamic Computational Graphs-It is more adaptable. The forward computation is implemented in preferred programming language. Debugging dynamic graphs is simple.

Encoder – decoder sequence to sequence architectures

Sequence-to-sequence learning (Seq2Seq) is about training models to convert sequences from one domain (e.g. sentences in English) to sequences in another domain (e.g. the same sentences translated to French). Recurrent neural network (RNN) is a popular sequence model that has shown efficient performance for sequential data.

"the cat sat on the mat" -> [Seq2Seq model] -> "le chat etait assis sur le tapis"

A sequence to sequence model lies behind numerous systems which you face on a daily basis. For instance, seq2seq model powers applications like Google Translate, voice-enabled devices and online chatbots.

Machine translation —



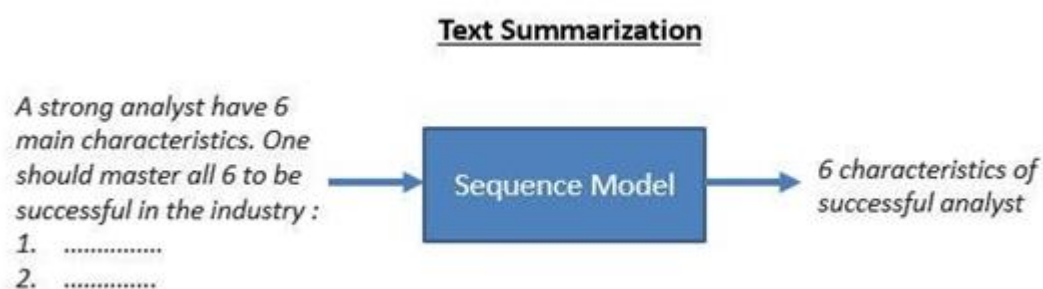
Speech recognition



Video captioning —on generating movie descriptions.



S2VT: A herd of zebras are walking in a field.



This model can be used as a solution to any sequence-based problem, especially ones where the inputs and outputs have different sizes and categories.

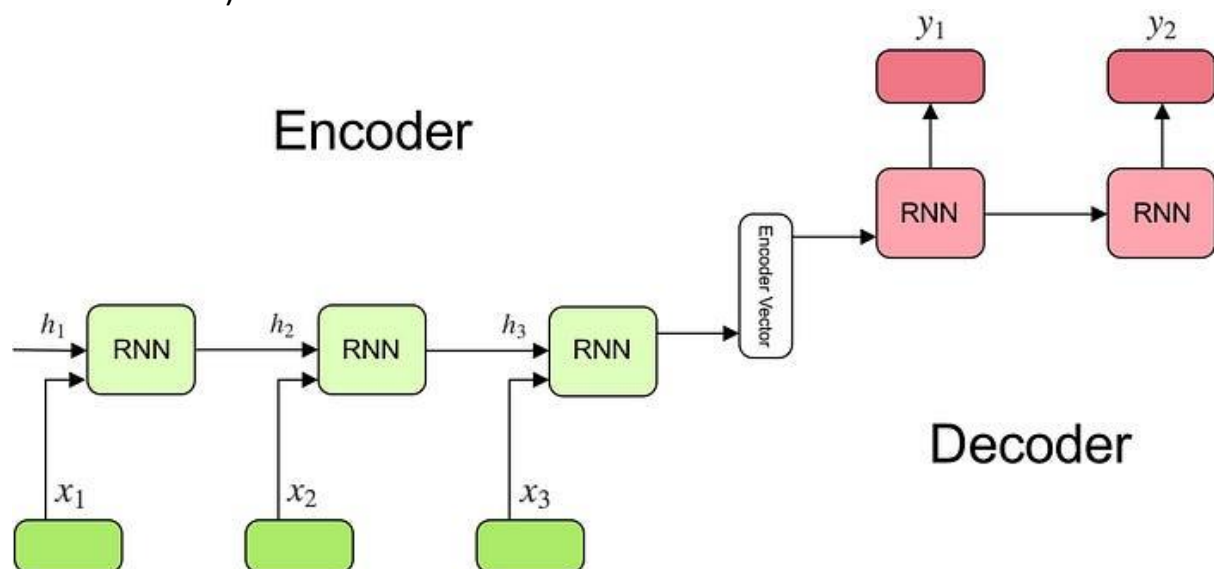
A sequence to sequence model aims to map a fixed-length input with a fixed-length output where the length of the input and output may differ.

For example, translating “What are you doing today?” from English to Chinese has input of 5 words and output of 7 symbols (今天你在做什麼?). Clearly, we

can't use a regular LSTM network to map each word from the English sentence to the Chinese sentence.

Sequence to Sequence Model

The model consists of 3 parts: encoder, intermediate /encoder/hidden vector and decoder. Both encoder and the decoder are LSTM models or GRU (Gated recurrent unit) models.



The encoder will convert the input sequence into hidden vector. The decoder will convert the hidden vector into output sequence.

Encoder

- Multiple recurrent units are stacked together where each accepts a single element of the input sequence, collects information for that element and propagates it forward.
- *Data is read*, one sequence after the other. Thus if the input is a sequence of length 't', then it will be read in 't' time steps. At each time step, the hidden state h will be updated with previous hidden state and the current input.
- X_i = Input sequence at time step i .
- h_i and c_i = 'h' for hidden state and 'c' for cell state. Combined together these are internal state at time step i .
- Y_i = Output sequence at time step i .
- After all the inputs are read by the encoder model, the final hidden state of the model represents the summary of the whole input sequence.

- In question-answering problem, the input sequence is a collection of all words from the question. Each word is represented as x_i where i is the order of that word.
- The hidden states h_i are computed using the formula:

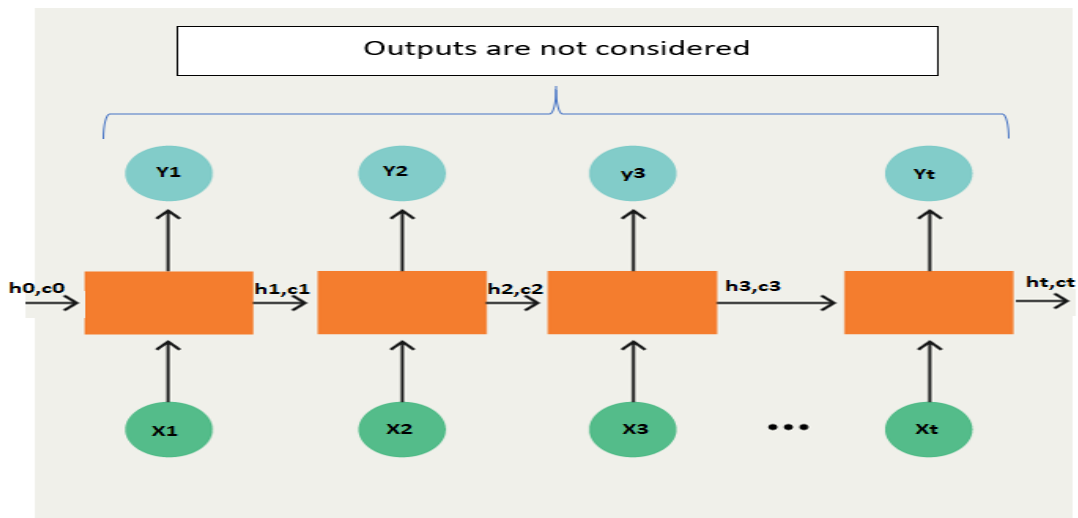
$$h_t = f(W^{(hh)}h_{t-1} + W^{(hx)}x_t)$$

W_{hh} - weight at the recurrent neuron

h_{t-1} - Previous State

W_{hx} - input neuron weight

x_t - current input



At the first time step t_1 , the previous hidden state h_0 will be considered zero. So the first RNN cell will update the current hidden state with the first input x_1 and h_0 .

Encoder Vector

- This is the final hidden state produced from the encoder part of the model.
- This vector aims to encapsulate the information for all input elements in order to help the decoder make accurate predictions.

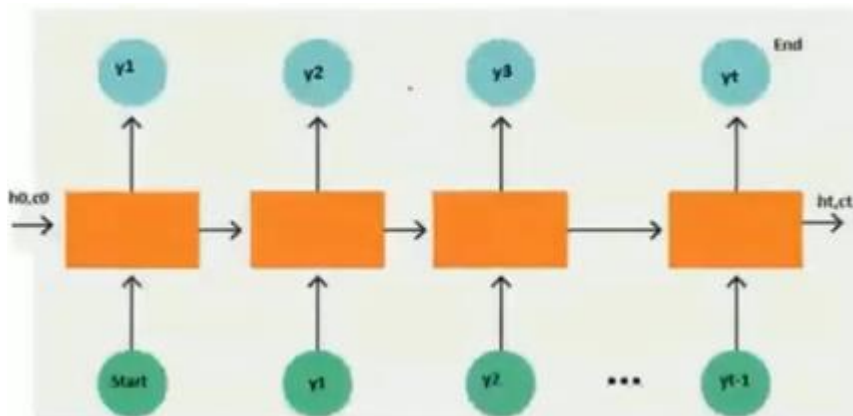
- It acts as the initial hidden state of the decoder part of the model.

Decoder

- A stack of several recurrent units where each predicts an output y_t at a time step t .
- Each recurrent unit accepts a hidden state from the previous unit and produces an output as well as its own hidden state.
- In the question-answering problem, the output sequence is a collection of all words from the answer. Each word is represented as y_i where i is the order of that word.
- Any hidden state h_i is computed using the formula:

$$h_t = f(W^{(hh)} h_{t-1})$$

The previous hidden state is used to compute the next one.



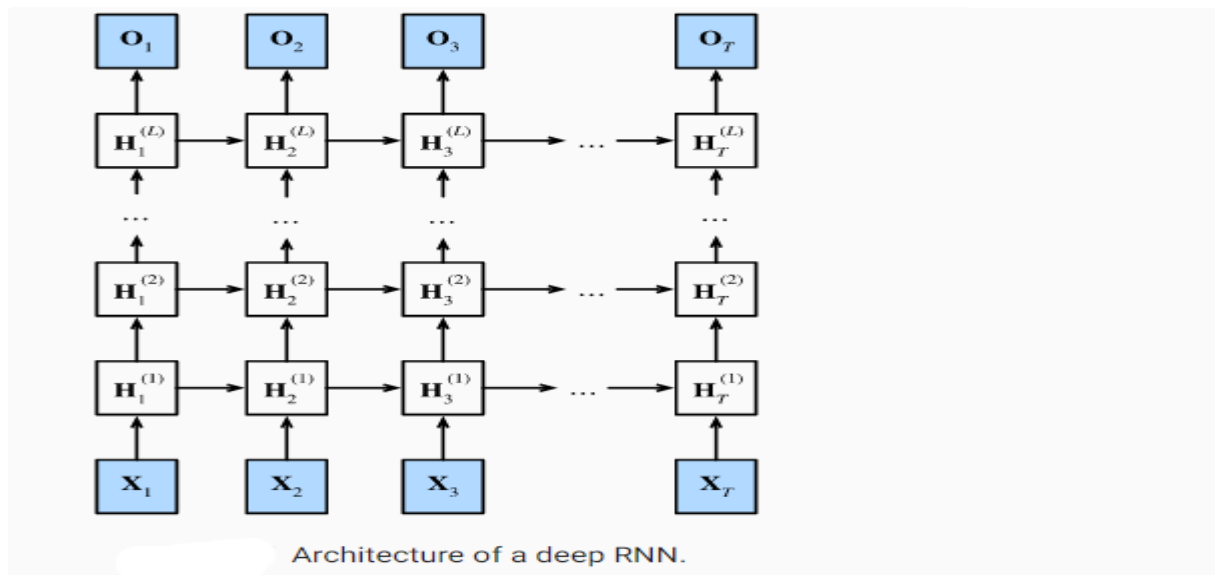
The output y_t at time step t is computed using the formula:

$$y_t = \text{softmax}(W^S h_t)$$

Deep recurrent networks

An RNN can also be made deep by introducing depth to a hidden unit. Inputs from the first time step can influence the outputs at the final time step T . These inputs pass through T applications of the recurrent layer before reaching the final output. The standard method for building deep RNN is stacking the

RNNs on top of each other. Given a sequence of length T , the first RNN produces a sequence of outputs, also of length T . These, in turn, constitute the inputs to the next RNN layer. In the figure given below, a deep RNN with L hidden layers is shown. Each hidden state operates on a sequential input and produces a sequential output. Moreover, any RNN cell (white box) at each time step depends on both the same layer's value at the previous time step and the previous layer's value at the same time step.



The hidden state in a deep RNN can be given by the following equation:

$$H_t^l = \phi^l(H_{t-1}^l * W_{hh}^l + H_t^{l-1} * W_{xh}^l + b_h^l)$$

Where ϕ is the activation function, W is the weight matrix, and b is the bias.

The output at a hidden state H_t is given by:

$$O_t = H_t^l * W_{hy} + b_y$$

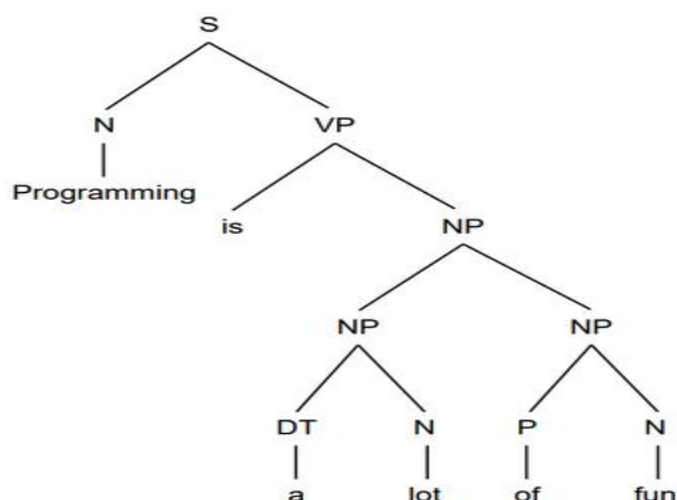
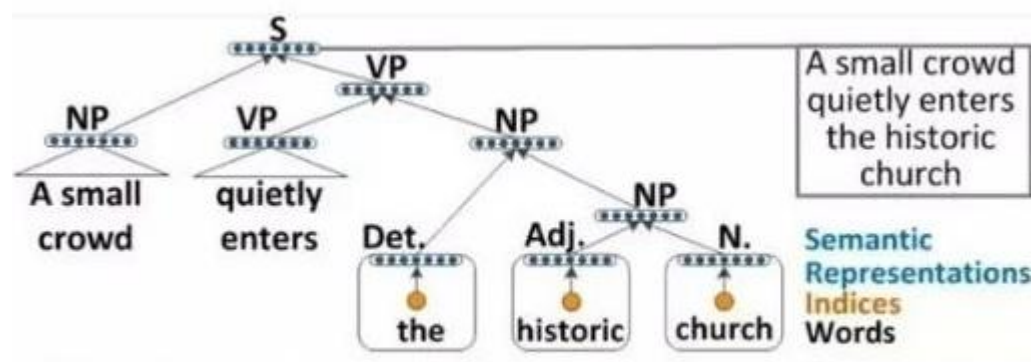
Recursive neural networks

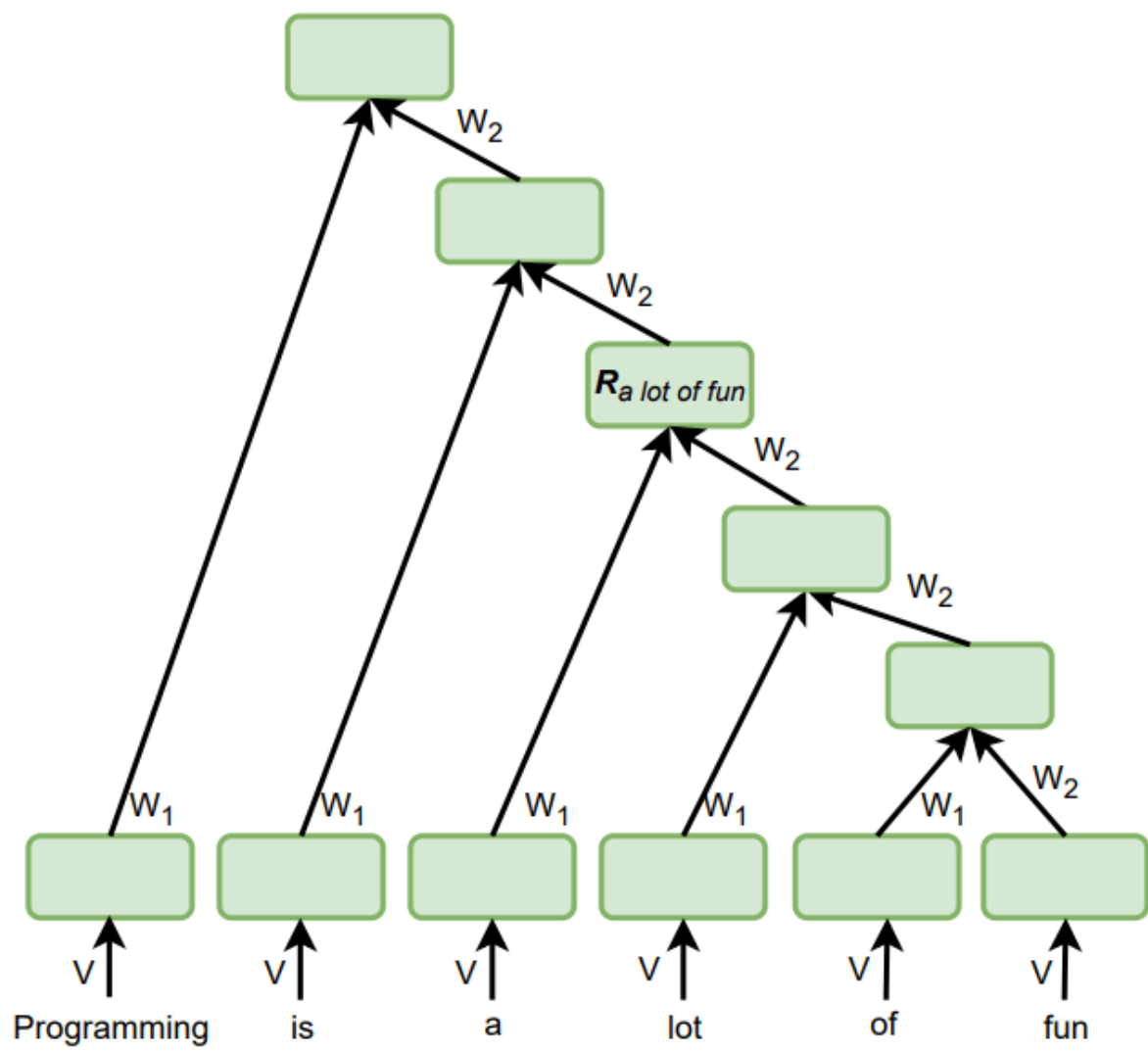
Recursive Neural Networks (RvNNs) are deep neural networks used for natural language processing. In Recursive Neural Network, the same weights are applied recursively on a structured input to obtain a structured prediction. The word recursive indicates that the neural network is applied to its output.

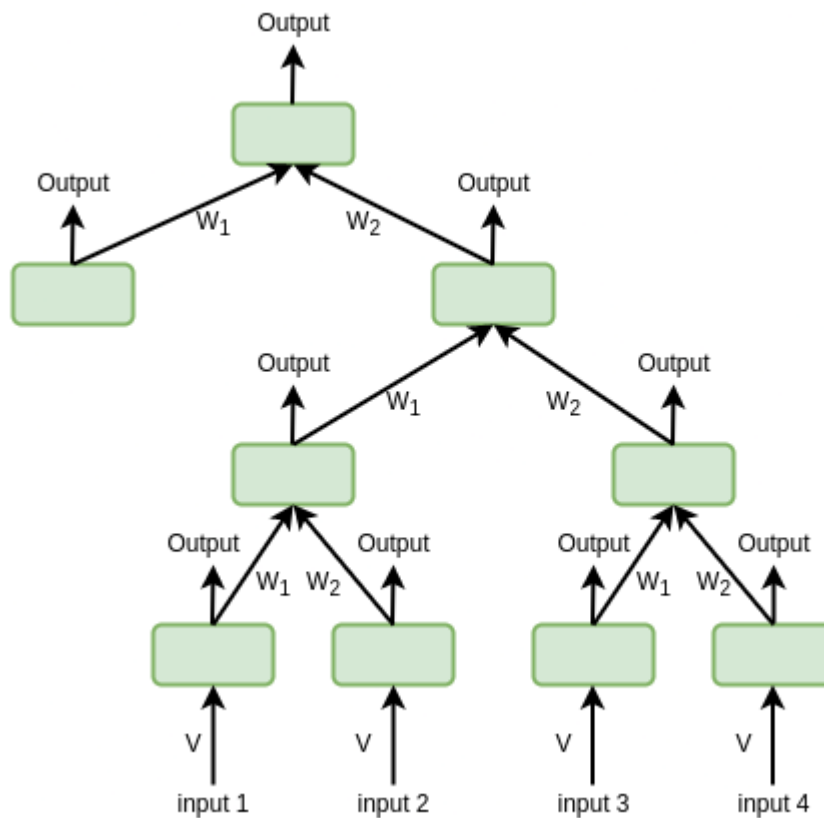
Due to their deep tree-like structure, Recursive Neural Networks can handle hierarchical data. The tree structure means combining child nodes and producing parent nodes. Each child-parent bond has a weight matrix, and similar children have the same weights. The number of children for every node in the tree is fixed. RvNNs are used when there's a need to parse an entire sentence.

A Recursive Neural Network is used for sentiment analysis in sentences. Sentiment analysis of sentences is among the major tasks of NLP (Natural Language Processing), that can identify writers writing tone & sentiments in any specific sentences. When a writer expresses any sentiments, basic labels around the tone of writing are identified. For instance, whether the meaning is a constructive form of writing or negative word choices.

A variable called 'score' is calculated at each traversal of nodes, telling us which pair of phrases and words we must combine to form the perfect syntactic tree for a given sentence.







Considering a binary tree, all the right children share one weight matrix and all the left children share another weight matrix. In addition, we need an initial weight matrix V to calculate the hidden state for each raw input

To calculate the parent node's representation, sum the products of the weight matrices W_i and the children's representations C_i and applying the transformation f

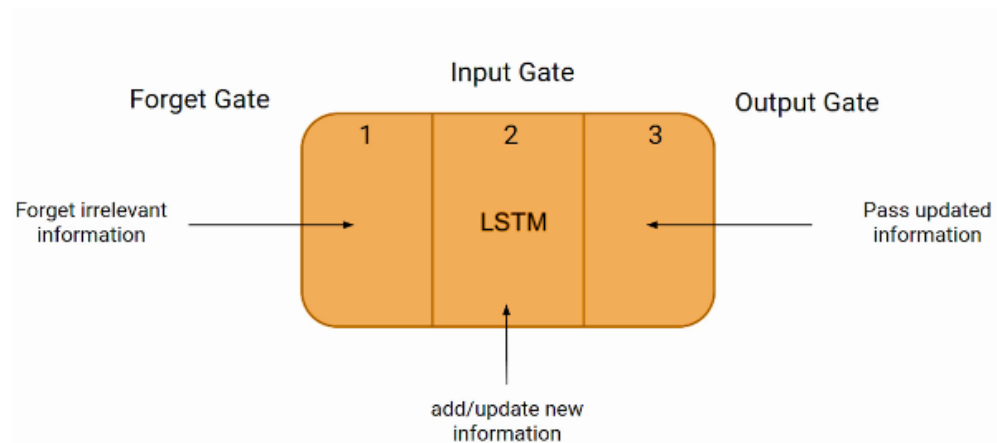
$$h = f \left(\sum_{i=1}^{i=c} W_i C_i \right)$$

Where c is the number of children.

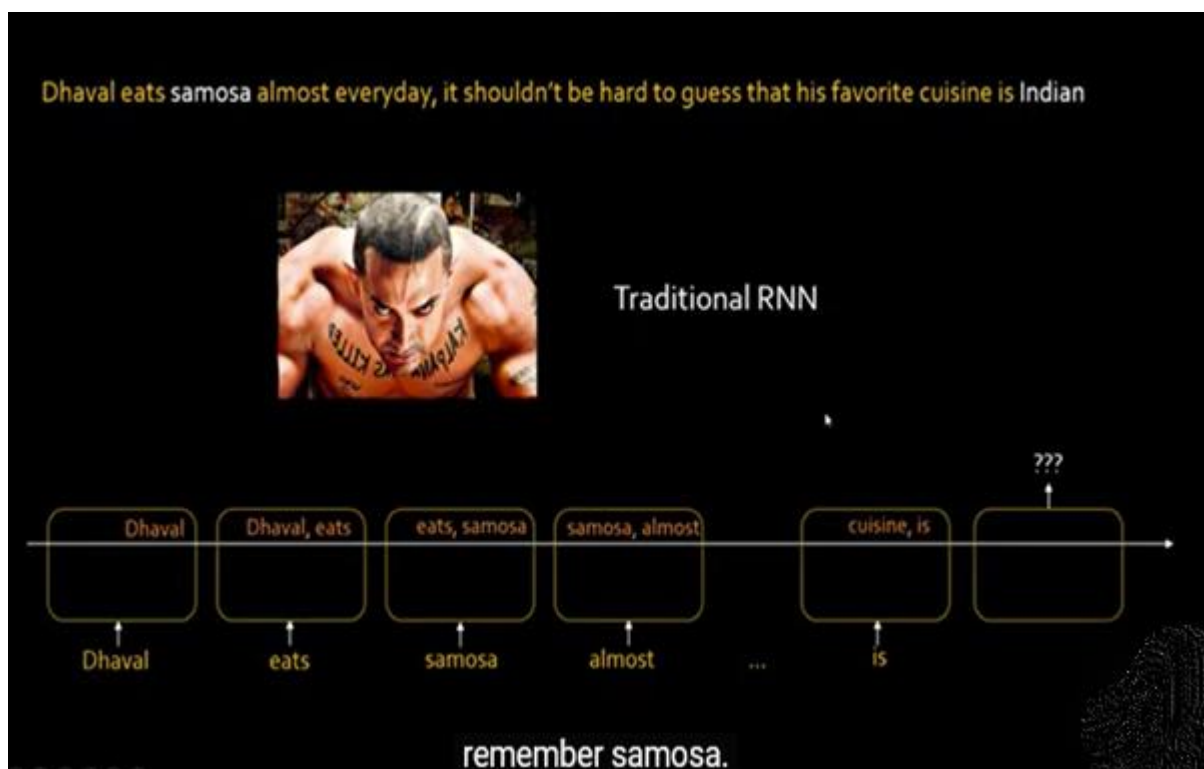
Long Short-Term Memory(LSTM)

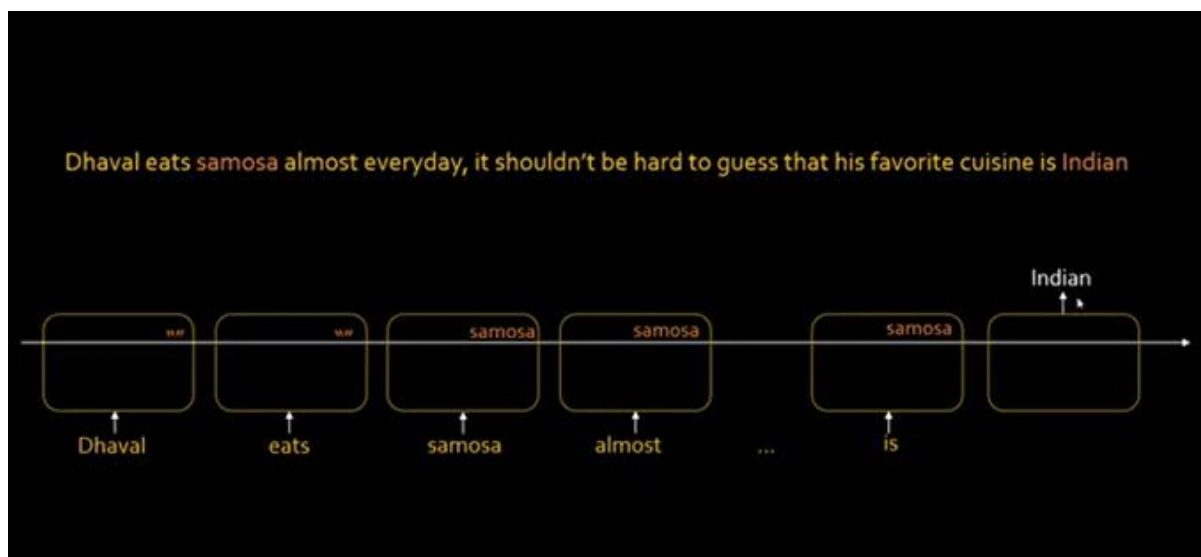
Long Short-Term Memory Networks is a deep learning, sequential neural network that allows information to persist. An LSTM unit that consists of three gates and a memory cell. The gates control the flow of information in and out of the memory cell. The first gate is called **Forget gate**, the second gate is

known as **the Input gate**, and the last one is **the Output gate**. The first part chooses whether the information coming from the previous timestamp is to be remembered or irrelevant and can be forgotten. In the second part, the cell tries to learn new information from the input to this cell. At last, in the third part, the cell passes the updated information from the current timestamp to the next timestamp.

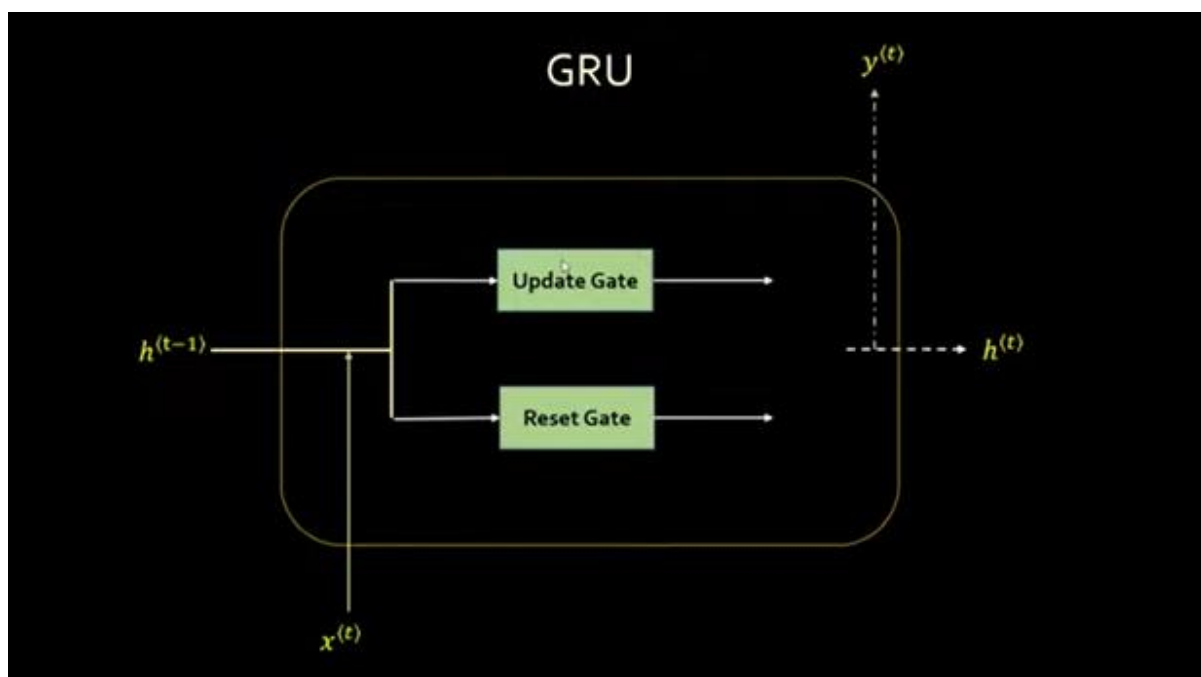


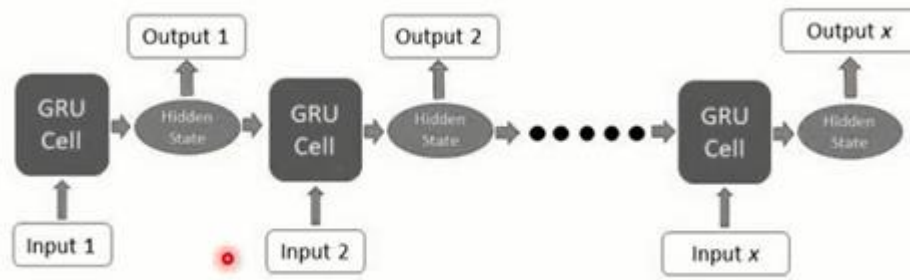
Gated Recurrent Unit (GRU)





GRU was invented in 2014 by K.Cho. Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN). Like LSTM, GRU can process sequential data. GRU has only a hidden state h_t (No cell state). Due to the simpler architecture, GRUs are faster to train. To solve the Vanishing-Exploding gradients problem (The lower the gradient is, the harder it is for the network to update the weights and the longer it takes to get to the final result.) encountered during Recurrent Neural Network, variations like **Long Short Term Memory Network (LSTM)** and **Gated Recurrent Unit Network (GRU)**.





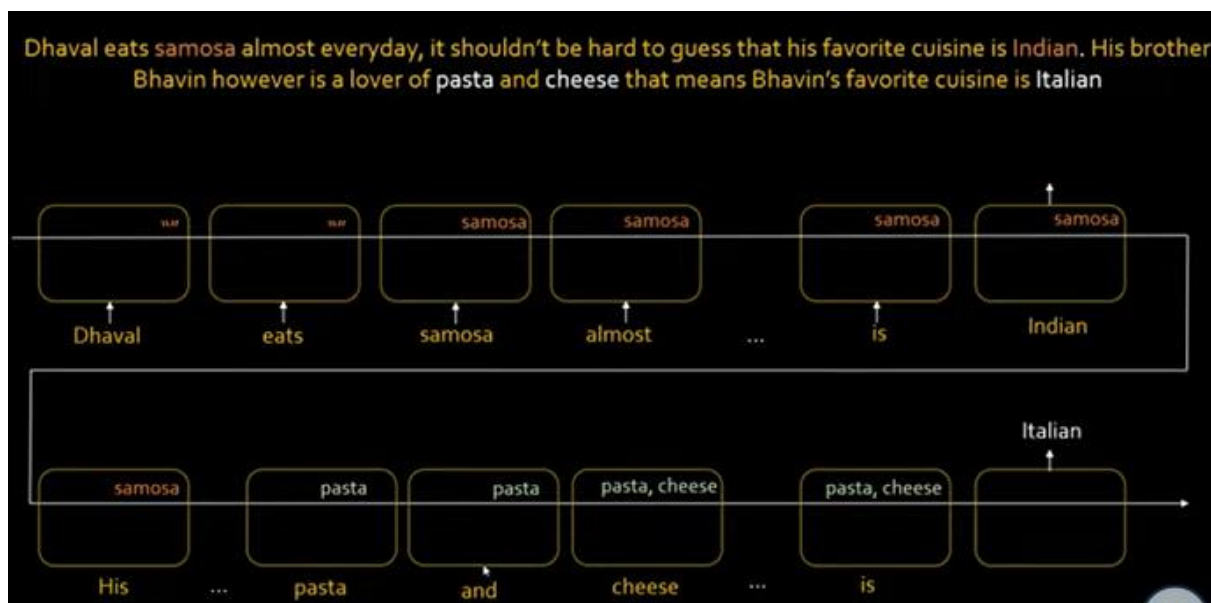
At each timestamp t , it takes an input X_t and the hidden state h_{t-1} from the previous timestamp $t-1$. It outputs a new hidden state h_t which again passed to the next timestamp.

The basic idea behind GRU is to use gating mechanisms to selectively update the hidden state of the network at each time step. The gating mechanisms are used to control the flow of information in and out of the network. The GRU has two gating mechanisms-reset and update gate.

Reset gate(r)- determines how much of the past information should be forgotten

Update gate(z)- determines how much of the past information should be maintained.

The output of the GRU is calculated based on the updated hidden state.



Reset gate is used to decide how much of the past information to forget. To

$$r_t = \sigma(W^{(r)}x_t + U^{(r)}h_{t-1})$$

Update gate z_t for time step t is calculated **using the formula**:

$$z_t = \sigma(W^{(z)}x_t + U^{(z)}h_{t-1})$$

When x_t is plugged into the network unit, it is multiplied by its own weight $W(z)$. The same goes for h_{t-1} which holds the information for the previous $t-1$ units and is multiplied by its own weight $U(z)$. Both results are added together and a sigmoid activation function is applied to squash the result between 0 and 1.

Current memory content(candidate hidden state)

Here a new memory content is introduced which will use the reset gate to store the relevant information from the past. It is calculated as follows:

$$h'_t = \tanh(Wx_t + r_t \odot Uh_{t-1})$$

1. Multiply the input x_t with a weight W and h_{t-1} with a weight U .
2. Calculate the Hadamard (element-wise) product between the reset gate r_t and Uh_{t-1} . That will determine what to remove from the previous time steps.

Consider a sentiment analysis problem for determining one's opinion about a book from a review he wrote. The text starts with "This is a fantasy book

which illustrates...” and after a couple of paragraphs ends with “I didn’t quite enjoy the book because I think it captures too many details.” To determine the overall level of satisfaction from the book we only need the last part of the review. In that case as the neural network approaches to the end of the text it will learn to assign r_t vector close to 0, washing out the past and focusing only on the last sentences.

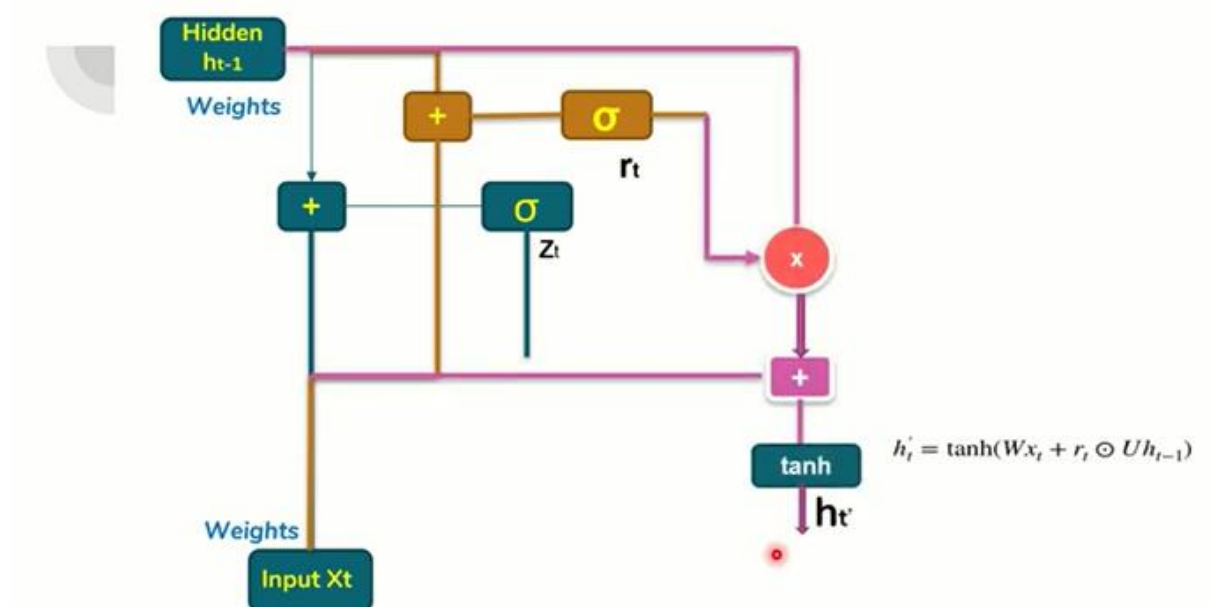
Eg:

You are reviewing a movie. Initially you start with “ the movie is directed by X, it has featured Y, Music by Z, etc.. After about 10 lines, you say, the movie, I think is a bad one for the money I paid”.

So, which is the actual review? The final line. So, the neural network should not remember the past sentences and focus on the last sentence to get the crux. It is what is enabled here.

Here, r_t will be assigned 0 until the final phrase is in analyze so as to drop the unfocussed items.

3. Sum up the results of step 1 and 2.
4. Apply the activation function $\tanh$.



Final memory at current time step(hidden state)

As the last step, the network needs to calculate h_t , which holds information for the current unit and passes it down to the network. In order to do that the update gate is needed. It determines what to collect from the current memory content — h'_t and what from the previous steps — $h_{(t-1)}$. That is done as follows:

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot h'_t$$

1. Apply element-wise multiplication to the update gate z_t and $h_{(t-1)}$.
2. Apply element-wise multiplication to $(1 - z_t)$ and h'_t .
3. Sum the results from step 1 and 2.

The main difference between the RNN and GNU is in the internal working within each recurrent unit as Gated Recurrent Unit networks consist of gates which modulate the current input and the previous hidden state.